MODULE 2 · LECTURE 10

CS 6530 Applied Cryptography

MAC, HMAC, KDF and Password Protection

Module 2 – Symmetric Cryptography, Hashes and Authentication

Why Do We Need a Secret?

Ordinary transmission — no attacker



Active attacker on the path — Eve intercepts and modifies M



Why does this fail?

- The hash function H(·) is public.
- No secret is needed to compute the digest.
- An attacker can modify M and recompute its hash.

Think Like an Engineer

Should message authentication depend on keeping the algorithm secret—or on keeping a key secret?.

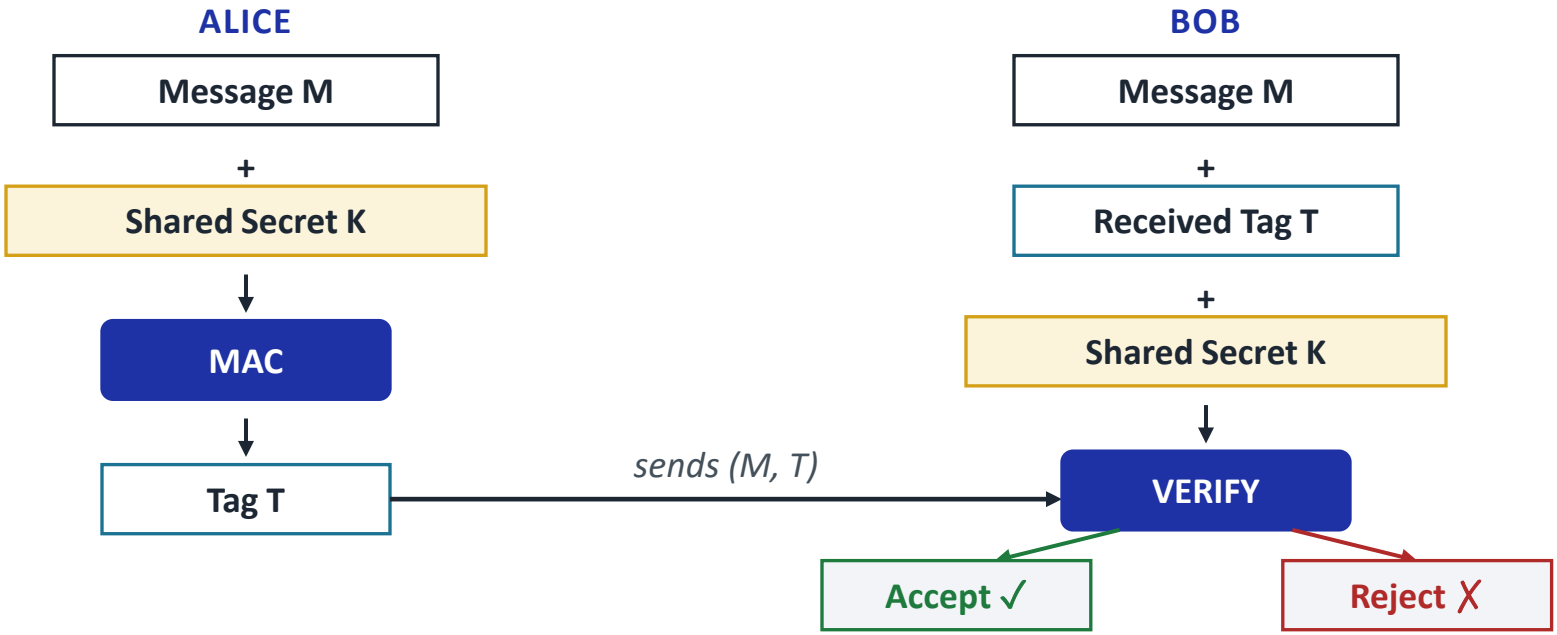
The cryptographic algorithm can be public; security should depend on the secret key.

Authentication requires something the attacker cannot compute — a value dependent on a secret.

A plain hash attached to a message does not provide message authentication.

SECRET-KEY MESSAGE AUTHENTICATION

MESSAGE AUTHENTICATION CODE (MAC)



*Shared secret K is established beforehand — it is **not** transmitted with (M, T).*

$T = MAC_K(M)$ $\quad$ $Verify_K(M, T) \rightarrow \{Accept, Reject\}$

✓ Integrity

Unauthorized modification of the message causes MAC verification to fail.

✓ Data-Origin Authentication

A valid tag indicates that it was generated by a holder of K

✗ Confidentiality

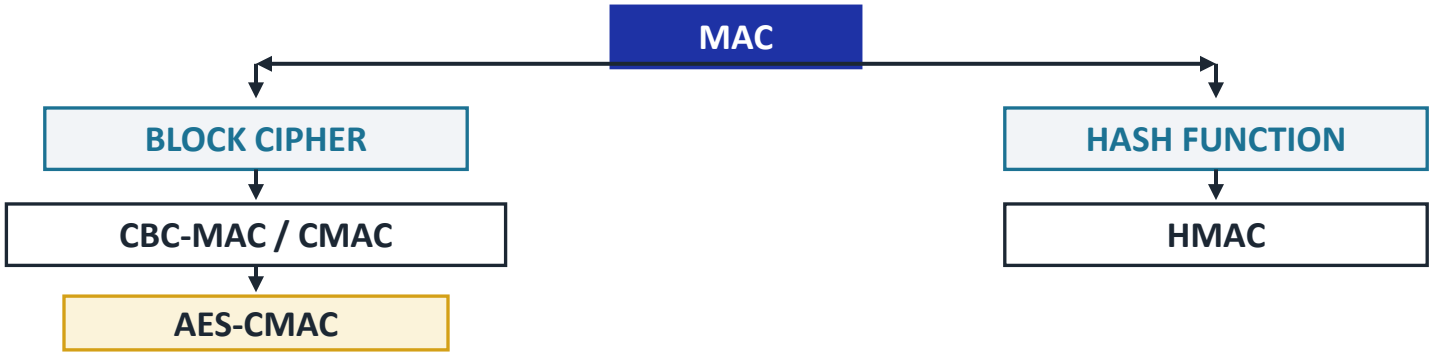
Message M may still be transmitted in plaintext.

✗ Non-repudiation

Either key holder could have generated the tag — a MAC cannot prove which one.

A MAC uses a shared secret to provide message integrity and data-origin authentication.

Block Cipher or Hash Function?



CMAC and HMAC are standardized MAC constructions built from different underlying primitives..

CBC-MAC: Block-Cipher Chaining

$C_1 = EK(M_1),\ C_i = EK(M_i \oplus C_{i-1})$

$T = C_n$

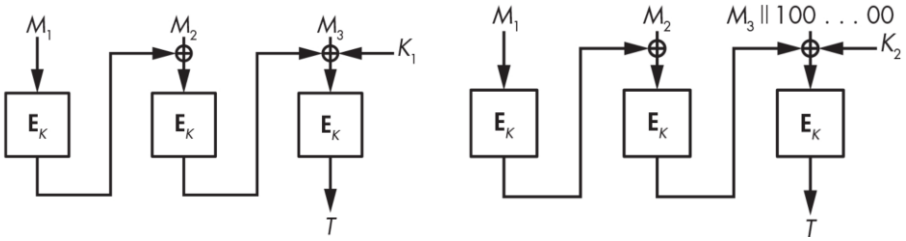
The final chaining value becomes the tag; M is not encrypted for transmission.

CBC Encryption → transmit $C_1 \ldots C_n$

CBC-MAC → transmit $M_1 \ldots M_n$ + Tag T

CBC-MAC requires care for variable-length messages; CMAC standardizes this.

CMAC — Handling the Final Block



Complete block → K_1 **Padded block → K_2**

CMAC treats complete and padded final blocks differently, using derived subkeys K_1 and K_2.

Hash Function → **HMAC** A standardized MAC built from a hash function and a secret key.

MAC is the security functionality; CMAC and HMAC are different standardized constructions for providing it.

Why Not Just H(K ‖ M)?

$$T = H(K \,\|\, M)$$

K = shared secret

M = message, T = proposed tag

Hash is secure ✓

Key K is secret ✓

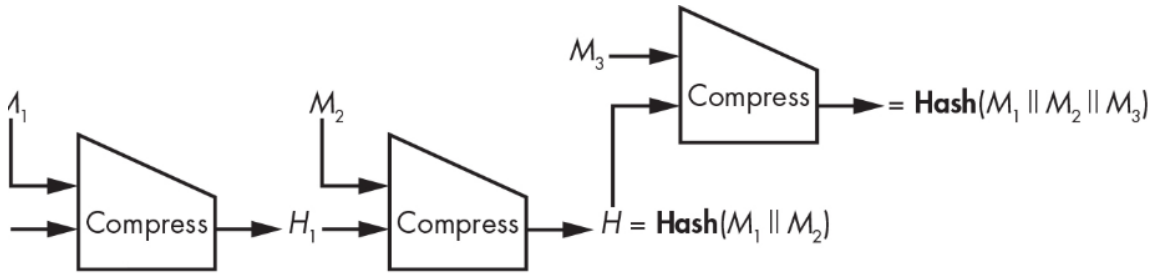
Tag depends on K, M ✓

Secure MAC? NOT AUTOMATICALLY

A secure cryptographic primitive does not automatically make every construction built from it secure.

Concrete Example — Length Extension

For Merkle–Damgård hashes (e.g., SHA-256), H(K‖M) can be susceptible to length extension.



The published digest reflects a chaining state from which processing can continue.

H(K‖M) → chaining state exploited → **valid tag for a related message with appended data**

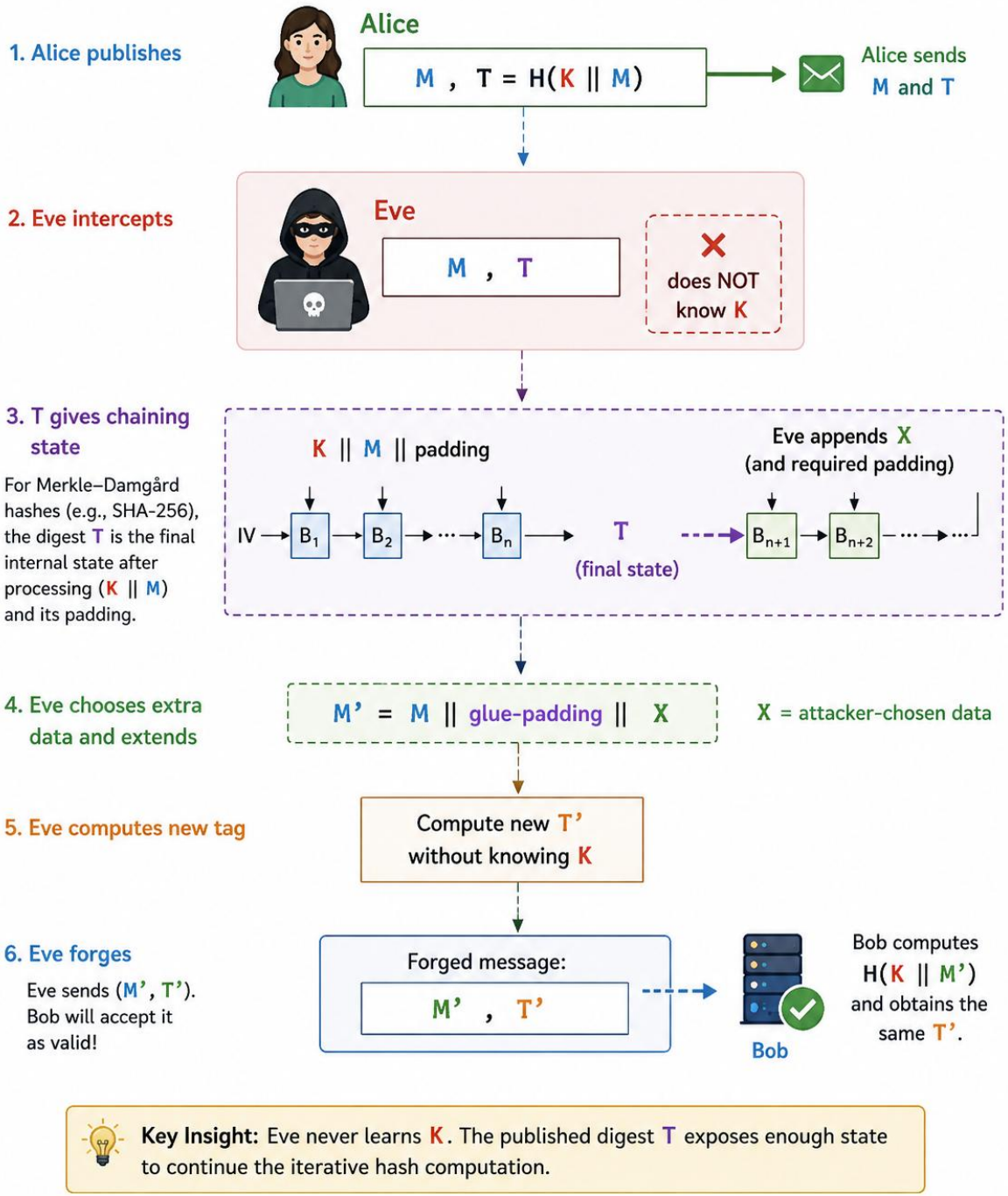
SHA-256 is not broken. The problem is insecure, ad-hoc use as H(K‖M).

HMAC A standardized MAC construction using a hash function and a secret key.

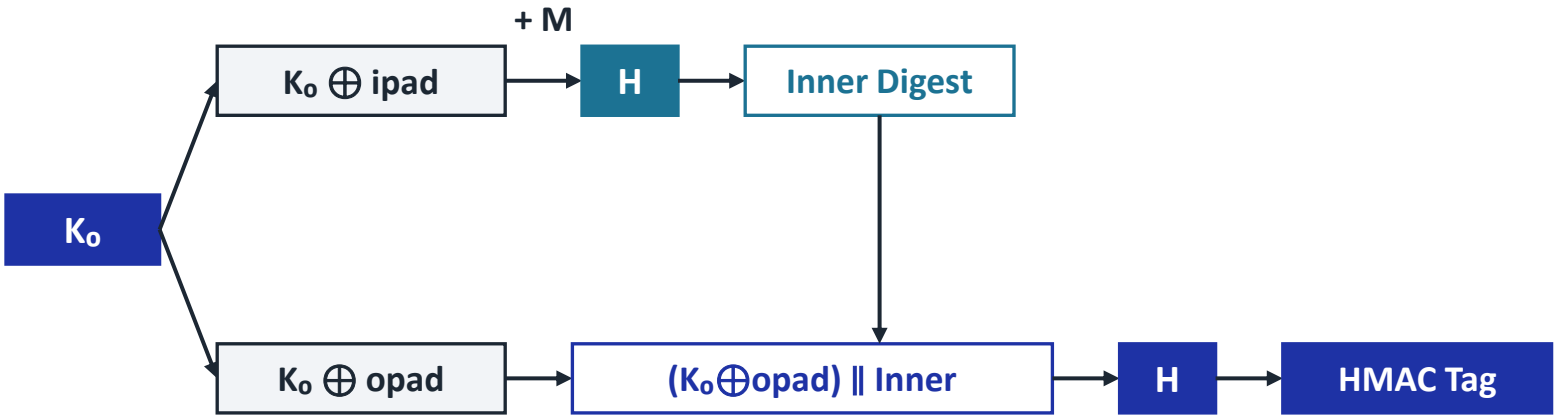
Secure hash ≠ arbitrary secure MAC construction. Use HMAC.

LENGTH-EXTENSION ATTACK

How Can Eve Extend the Message Without Knowing K?



HMAC — Keyed-Hash Message Authentication Code



$$HMAC(K,M) = H((K_o \oplus opad) \| H((K_o \oplus ipad) \| M))$$

Key Preparation

B = input block size of the hash. HMAC builds a block-sized internal key K_o:

$|K| > B \rightarrow K_o = H(K)$, zero-padded to B
$|K| < B \rightarrow K_o = K$, zero-padded to B
$|K| = B \rightarrow K_o = K$

Zero-padding is normalization — it adds no entropy.

HMAC-SHA-256

Secret key K	→ not fixed to 64 bytes
SHA-256 block B	→ 64 bytes (512 bits)
SHA-256 output L	→ 32 bytes (256 bits)

Key length ≠ Block size ≠ Output size

HMAC Is Generic — HMAC-SHA-256, HMAC-SHA-512, HMAC-SHA3-256, HMAC-SHA3-512. SHA-256 is the example used here; HMAC is not restricted to SHA-256 or SHA-2.

Specification: RFC 2104; NIST FIPS 198-1 (NIST has proposed transitioning guidance to SP 800-224).

HMAC uses a normalized secret key with inner and outer hash computations to construct a MAC.

Modify the Message — Verification Fails

M = `"Transfer ₹10,000 to Account B"` — Alice and Bob share secret key K.



Secret key K is not transmitted.

Attack Scenario — Eve on the Path

Eve changes M → M′

`₹10,000` → `₹90,000` *M′ = modified message*

Eve knows M′ ✓ Eve knows HMAC-SHA-256 ✓ Eve does NOT know K ✗

`Cannot generate valid HMAC-SHA-256(K, M′)` → Bob: **INVALID → REJECT**

Plain Hash	HMAC
H(M′)	$HMAC_k(M')$
Anyone can recompute	Requires secret K
No message authentication	Shared-secret authentication

Security Boundary — HMAC authenticates the message; it does not encrypt it. M may remain visible in transit. For confidentiality + authentication, use an AEAD scheme.

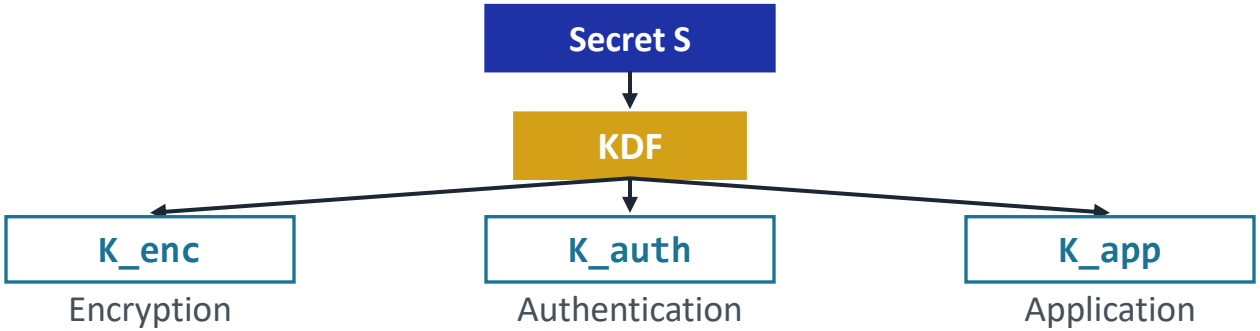
Without the secret key, modifying the message does not allow the attacker to generate the corresponding valid HMAC tag.

FROM ONE SECRET TO MANY KEYS

KDF and Key Separation

What Is a KDF?

KDF = Key Derivation Function — cryptographically derives one or more keys from source keying material.



`K_enc = K_auth = K_app = S`
`implies Key reuse ✗`

`K_enc, K_auth, K_app` **Key separation ✓**

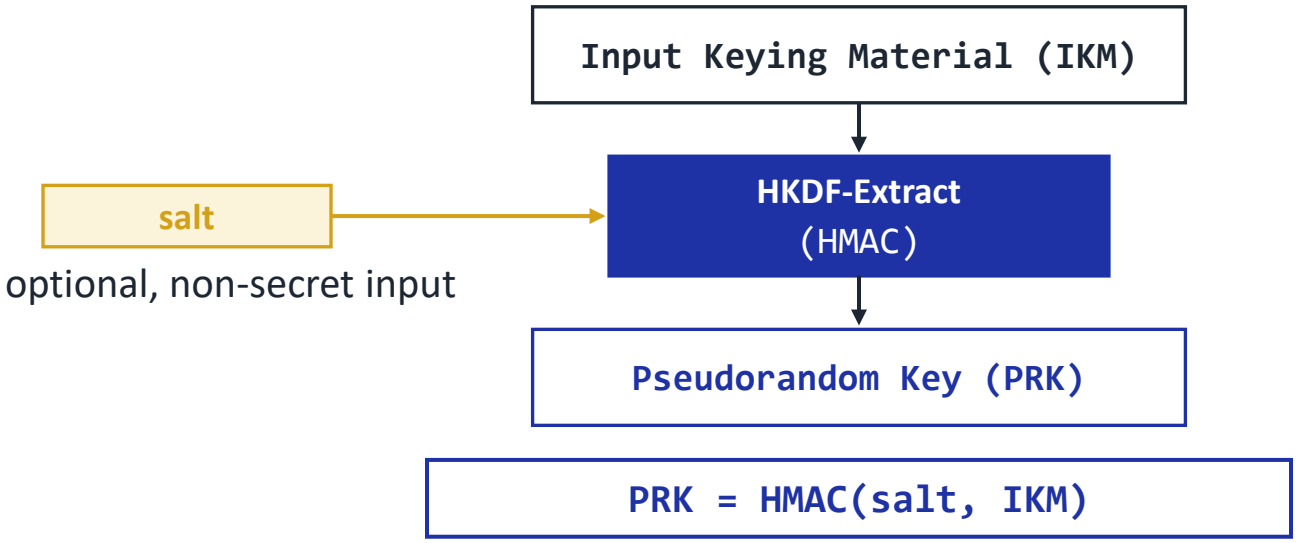
`Secret Keying Material + Context → KDF → Derived Key(s)`

HKDF = HMAC-based Extract-and-Expand Key Derivation Function

`Hash → HMAC → HKDF`

Derive separate keys for separate cryptographic purposes instead of reusing one secret everywhere.

Extract — Obtain a Pseudorandom Key



For HKDF-SHA-256: |PRK| = 32 bytes

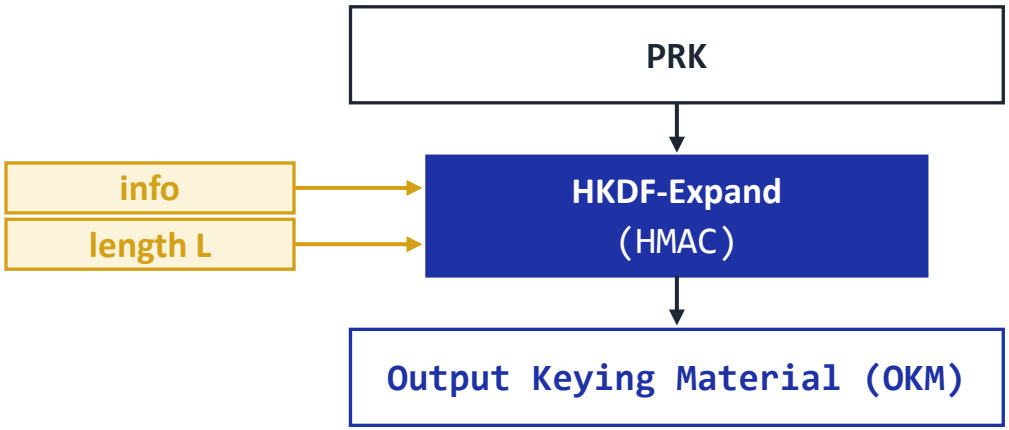
Extract concentrates the useful entropy in the input keying material into a fixed-length pseudorandom key.

HKDF does not create entropy. If the input keying material is weak or predictable, HKDF cannot turn it into a strong secret.

The HKDF salt need not be secret.

HKDF-Extract uses IKM and an optional, non-secret salt to produce a fixed-length pseudorandom key (PRK)

Expand — Derive the Key Material We Need



`OKM = HKDF-Expand(PRK, info, L)`

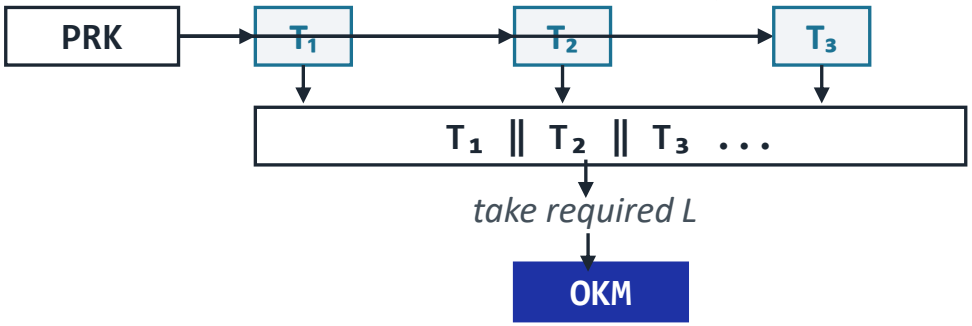
PRK
Pseudorandom Key

`info`
context information

L
required output length

OKM
Output Keying Material

How Can It Produce More Than One Hash Output?



For the first output block: $T_1 = HMAC_{PRK}(info \| 0x01)$

For the second: $T_2 = HMAC_{PRK}(T_1 \| info \| 0x02)$,

then: $T_3 = HMAC_{PRK}(T_2 \| info \| 0x03)$

and so on.

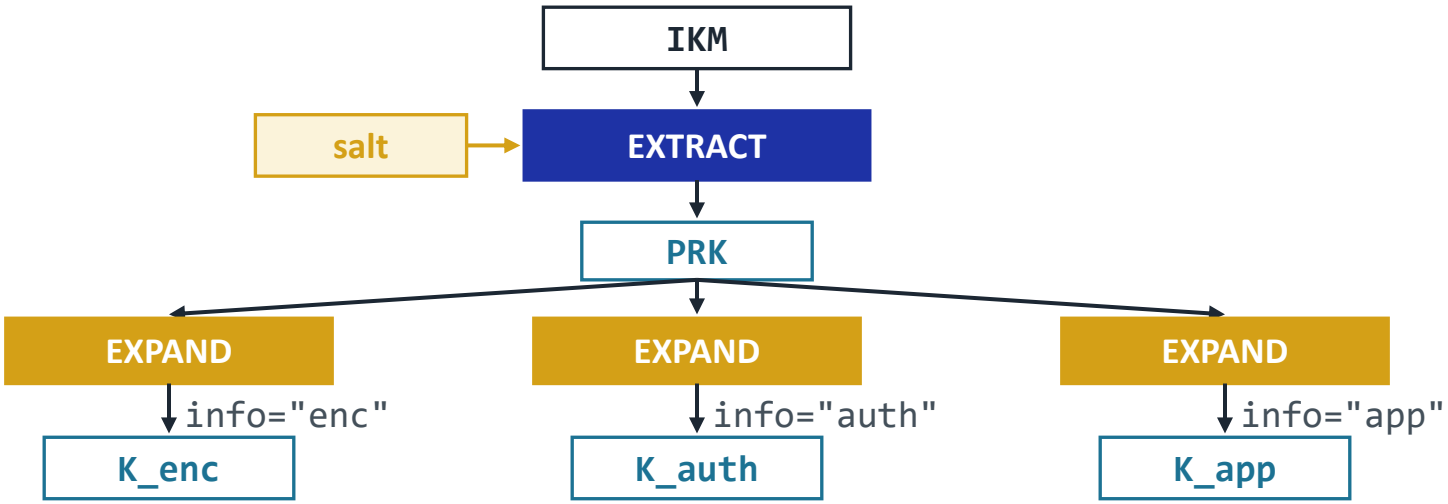
Finally: $OKM = T_1 \| T_2 \| T_3$ ... truncated to the length L

HKDF-Expand generates successive HMAC output blocks until enough keying material is available.

Extract produces PRK; Expand produces usable output keying material.

Expand uses the PRK, context and required length to derive output keying material.

Context Gives Us Key Separation




```
K_enc = Expand(PRK, "enc" ,Lenc); K_auth = Expand(PRK, "auth", Lauth);
K_app = Expand(PRK, “app“ ,Lapp)
```

Role of info — binds derived key material to an application, purpose, or protocol context (enc vs auth, protocol purpose, session context).

different info (context) → different derived key material.

info is not secret.
It provides context binding, not secrecy.

HKDF is used in modern secure protocols, including TLS 1.3.

EXTRACT
`IKM + salt → PRK`

EXPAND
`PRK + info + L → OKM`

One underlying secret can safely support different purposes by deriving cryptographically separated keys with appropriate context.

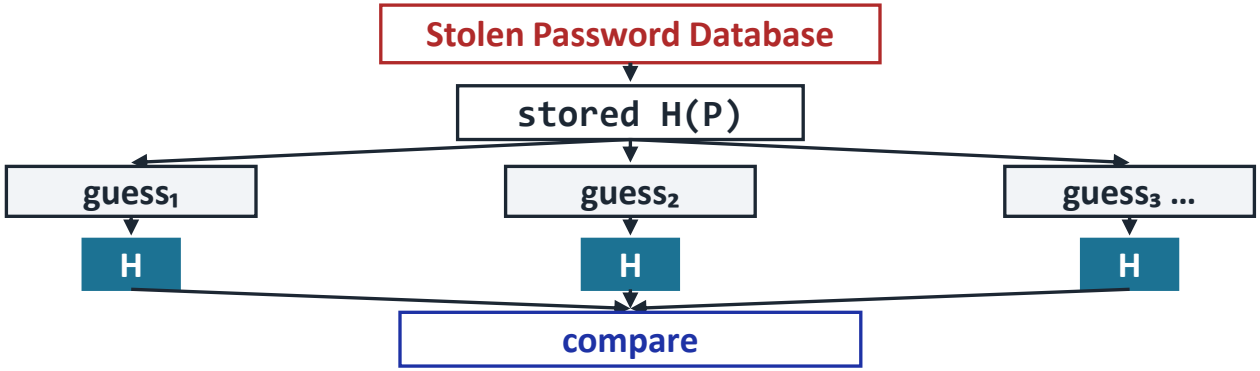
Offline Attacks, Salt and Work Factor

Crypto Key → random, high entropy

Human Password → chosen, low entropy

Passwords are not cryptographic keys.

`Stored verifier = H(P)` Login: `P' → H(P') =? H(P)`



OFFLINE ATTACK

Tested on attacker hardware — no server contact.

`Online guessing`
Server can rate-limit attempts.

`Offline (after DB theft)`
Attacker controls rate and hardware.

SHA-256 is designed to be fast — fast verification also means fast attacker guessing (not a SHA-256 flaw).

`Stored record = (S, F(P, S))`

Salt: unique/random per record · not secret · stored with the verifier

Same password P: $P+S_1 \rightarrow F \rightarrow A$ $P+S_2 \rightarrow F \rightarrow B$ $A \neq B$

Salt does not make a weak password strong.

`P + S` → `Password Hashing (Work Factor ↑)` → `Verifier`

The legitimate user pays the cost once; an attacker pays it repeatedly across many guesses.

Password storage must resist offline guessing: use a unique salt and deliberately costly password hashing.

PASSWORD HASHING AND DESIGN CHOICE

Make Guessing Expensive — Then Choose the Right Primitive

`Password + Salt → (time + memory cost) → Verifier`

Millions of guesses → Parallel HW (GPU/specialized) → Time + Memory Cost → More expensive large-scale guessing

Memory-hard password hashing requires significant memory as well as computation for each guess.

Security goal: memory hardness raises the cost of highly parallel password cracking.

Argon2id	scrypt	PBKDF2
Modern memory-hard password hashing	Memory-hard password-based derivation	Configurable work factor; not memory-hard

Choose the Right Tool

HASH	HMAC	HKDF	PASSWORD HASH
No secret → designed to be fast	Secret key → message authentication	Secret keying material → derive separated keys	Password + salt → deliberately costly

Think Like an Engineer

Need a cryptographic key from a human password — use HKDF directly?

No — first use a password-based derivation mechanism designed for low-entropy input.

Cryptographic primitives are not interchangeable — security depends on using the right construction for the right purpose

Standards and Recommended Reading

CMAC	→	NIST SP 800-38B
HMAC	→	RFC 2104 · NIST FIPS 198-1
HKDF	→	RFC 5869
Argon2	→	RFC 9106

Standards

CMAC
NIST SP 800-38B
Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication

HMAC
RFC 2104 — Krawczyk, Bellare, Canetti
HMAC: Keyed-Hashing for Message Authentication
NIST FIPS 198-1
The Keyed-Hash Message Authentication Code
NIST has proposed transitioning this guidance to SP 800-224.

HKDF
RFC 5869 — Krawczyk, Eronen
HMAC-based Extract-and-Expand Key Derivation Function (HKDF)

Password-Based Cryptography
RFC 9106
Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications
NIST SP 800-132
Recommendation for Password-Based Key Derivation

Textbooks / Further Reading

Aumasson, *Serious Cryptography, 2nd Ed.* — cryptographic hashing, MACs, secure construction use.
Stallings — message authentication, MACs, hash-based constructions.
Paar & Pelzl, *Understanding Cryptography* — hash functions, message authentication.

Hash → MAC / HMAC → KDF / HKDF → Password Protection

Use standardized cryptographic constructions — and choose the construction that matches the security problem.